



## RESERVAPP

**Autor:** Jordi Costa Mollà

**Curso:** 2º DAM

**Año:** 25/26

**Tutor del proyecto:** Víctor Pla Soler

**IES LA VEREDA**

**Resumen**

El proyecto ReservApp consiste en el desarrollo de una plataforma digital destinada a la gestión de servicios y reservas para pequeñas empresas y clientes.

Su objetivo es digitalizar y optimizar los procesos de organización de citas, facilitando la interacción entre empresas que ofrecen sus servicios y usuarios que desean contratarlo de forma rápida y centralizada.

La solución se compone de una aplicación web dirigida a las empresas, que permite registrarse, crear un perfil, publicar sus servicios, definir sus horarios de atención y gestionar las reservas.

Por otra parte, los clientes disponen de una aplicación móvil que les permite registrarse, buscar servicios y reservar citas de manera sencilla.

Desde el punto de vista técnico, el backend se desarrolla en Java utilizando el framework Spring Boot, implementando una API RESTful que da soporte tanto a la aplicación web como a la móvil.

El frontend web se construye con React y Vite, Bootstrap, JavaScript y CSS, mientras que la aplicación móvil se desarrolla con Flutter.

La base de datos relacional, implementada en SQL, gestiona la información de usuarios, empresas, servicios, horarios y reservas, garantizando la integridad y coherencia de los datos mediante un diseño normalizado.

Cada componente de la aplicación se despliega en su propio contenedor Docker, y el control de versiones se realiza con Git.

ReservApp busca ofrecer una solución moderna, intuitiva y escalable para la digitalización de pequeños negocios, contribuyendo a su transformación digital y mejorando la experiencia tanto de las empresas como de sus clientes.

**Resum**

El projecte ReservApp consisteix en el desenvolupament d'una plataforma digital destinada a la gestió de serveis i reserves per a petites empreses i clients.

El seu objectiu és digitalitzar i optimitzar els processos d'organització de cites, facilitant la interacció entre empreses que ofereixin els seus serveis i usuaris que desitgen contractar-lo de manera ràpida i centralitzada.

La solució es compon d'una aplicació web adreçada a les empreses, que permet registrar-se, crear un perfil, publicar els seus serveis, definir els seus horaris d'atenció i gestionar les reserves.

D'altra banda, els clients disposen d'una aplicació mòbil que els permet registrar-se, cercar serveis i reservar cites de manera senzilla.

Des del punt de vista tècnic, el backend és desenvolupa a Java utilitzant el framework Spring Boot, implementant una API RESTful que dona suport tant a l'aplicació web com a la mòbil.

El frontend web és construïx amb React i Vite, Bootstrap, JavaScript i CSS, mentre que l'aplicació mòbil es desenvolupa amb Flutter.

La base de dades relacional, implementada en SQL, gestiona la informació d'usuaris, empreses, serveis, horaris i reserves, garantint la integritat i coherència de les dades mitjançant un disseny normalitzat.

Cada component de l'aplicació es desplega en el seu propi contenedor Docker, i el control de versions es realitza amb Git.

ReservApp busca oferir una solució moderna, intuïtiva i escalable per a la digitalització de petits negocis, contribuint a la seva transformació digital i millorant l'experiència tant de les empreses com dels seus clients.

### **Abstract**

The ReservApp project consists of developing a digital platform for managing services and bookings for small businesses and customers.

Its aim is to digitise and optimise appointment scheduling processes, facilitating interaction between companies offering their services and users who want to hire them quickly and centrally.

The solution consists of a web application aimed at businesses, which allows them to register, create a profile, publish their services, define their opening hours and manage bookings.

On the other hand, customers have a mobile application that allows them to register, search for services and book appointments easily.

From a technical point of view, the backend is developed in Java using the Spring Boot framework, implementing a RESTful API that supports both the web and mobile applications.

The web frontend is built with React and Vite, Bootstrap, JavaScript and CSS, while the mobile application is developed with Flutter.

The relational database, implemented in SQL, manages user, company, service, schedule and booking information, ensuring data integrity and consistency through a standardised design.

Each component of the application is deployed in its own Docker container, and version control is performed with Git.

ReservApp seeks to offer a modern, intuitive, and scalable solution for the digitisation of small businesses, contributing to their digital transformation and improving the experience of both companies and their customers.

## Índice

|                                                  |    |
|--------------------------------------------------|----|
| 1. Introducción.....                             | 7  |
| 1.1. Módulos a los que implica.....              | 7  |
| 1.2. Descripción del proyecto.....               | 11 |
| 1.2.1. Visión general.....                       | 11 |
| 1.2.2. Objetivos técnicos.....                   | 11 |
| 2. Estudio previo.....                           | 11 |
| 2.1. Estudio de soluciones existentes.....       | 13 |
| 2.1.1. Soluciones analizadas.....                | 13 |
| 2.1.1.1. Calendly.....                           | 13 |
| 2.1.1.2. Booksy.....                             | 13 |
| 2.1.1.3. Setmore.....                            | 14 |
| 2.1.1.4. SimplyBook.....                         | 14 |
| 2.1.2. Conclusiones.....                         | 15 |
| 3. Plan de trabajo.....                          | 15 |
| 3.1. Fases del proyecto.....                     | 16 |
| 3.1.1. Análisis de requisitos.....               | 16 |
| 3.1.2. Diseño del sistema.....                   | 16 |
| 3.1.3. Implementación del backend.....           | 17 |
| 3.1.4. Desarrollo del frontend web.....          | 17 |
| 3.1.5. Desarrollo de la aplicación móvil.....    | 18 |
| 3.1.6. Integración y despliegue.....             | 18 |
| 3.1.7. Pruebas y validación.....                 | 18 |
| 3.1.8. Documentación y control de versiones..... | 19 |
| 4. Diseño.....                                   | 19 |
| 4.1. Diseño de la interfaz de usuario.....       | 19 |
| 4.2. Creación de mockups.....                    | 20 |
| 4.3. Herramientas de diseño.....                 | 21 |
| 4.4. Beneficios del prototipado.....             | 22 |
| 5. Implantación.....                             | 22 |
| 5.1. Implantación en el servidor.....            | 23 |
| 5.1.1. Desarrollo del backend.....               | 23 |
| 5.1.1.1. Estructura del backend.....             | 24 |
| 5.1.2. Configuración de la base de datos.....    | 27 |
| 5.1.3. Contenerización y despliegue.....         | 28 |
| 5.2. Implantación del frontend web.....          | 29 |
| 5.2.1. Entorno de desarrollo utilizado.....      | 29 |
| 5.2.2. Estructura del proyecto.....              | 30 |

|                                                       |    |
|-------------------------------------------------------|----|
| 5.2.3. Funcionamiento general.....                    | 32 |
| 5.2.4. Ventajas de esta arquitectura frontend.....    | 33 |
| 5.3. Implantación de la aplicación móvil.....         | 33 |
| 5.3.1. Entorno de desarrollo utilizado.....           | 33 |
| 5.3.2. Estructura del proyecto móvil.....             | 34 |
| 5.3.3. Cliente HTTP centralizado.....                 | 35 |
| 5.3.4. Funcionalidades principales implementadas..... | 36 |
| 5.3.5. Ventajas técnicas de Flutter.....              | 37 |
| 6. Análisis DAFO.....                                 | 37 |
| 6.1. Conclusiones del análisis.....                   | 38 |
| 7. Recursos.....                                      | 38 |
| 7.1. Herramientas hardware.....                       | 38 |
| 7.2. Herramientas software.....                       | 39 |
| 7.3. Sistemas operativos empleados.....               | 40 |
| 7.4. Paquetes y dependencias necesarias.....          | 40 |
| 8. Conclusiones.....                                  | 41 |
| 8.1. Grado de consecución de objetivos.....           | 42 |
| 8.2. Problemas encontrados.....                       | 43 |
| 8.3. Mejoras.....                                     | 43 |
| 9. Bibliografía.....                                  | 44 |

# 1. Introducción

En la actualidad la digitalización de los negocios se ha convertido en un factor clave para mejorar la eficiencia y la competitividad. ReservApp se crea como una solución tecnológica destinada a pequeñas empresas que deseen optimizar la gestión de sus servicios y reservas.

El proyecto consiste en el desarrollo de una plataforma digital que conecta a empresas y clientes mediante una aplicación web y una aplicación móvil, facilitando la organización de citas y la contratación de servicios de forma rápida, centralizada y accesible. Con una arquitectura moderna basada en Docker para su despliegue, ReservApp ofrece una herramienta escalable, intuitiva y orientada tanto a los negocios como a sus usuarios.

## 1.1. Módulos a los que implica

En este apartado se puede ver la influencia que han tenido todas las asignaturas del curso en el presente proyecto.

Asignaturas del primer curso:

**Sistemas informáticos** – El proyecto requiere la instalación, configuración y administración de entornos de desarrollo y ejecución tanto en local como en contenedores Docker.

Resultados de aprendizaje que se cumplen:

- RA 1: Evaluar y seleccionar los sistemas y recursos necesarios para ejecutar el backend, frontend y base de datos.
- RA 5: Configurar los contenedores de Docker y la conexión entre los servicios.

**Bases de datos** – El proyecto incluye el diseño e implementación de una base de datos relacional en MySQL para la gestión de empresas y clientes.

Resultados de aprendizaje que se cumplen:

- RA 2: Crear la base de datos definiendo su estructura y relaciones.
- RA 3: Consultar la información almacenada en la base de datos.
- RA 6: Diseñar el modelo relacional normalizado e interpretarlo en diagramas entidad/relación.

**Programación** – El backend, desarrollado en Java con Spring Boot, aplica los principios de la programación orientada a objetos para gestionar la lógica de negocio y la comunicación con la base de datos.

Resultados de aprendizaje que se cumplen:

- RA 1: Reconocer la estructura de la aplicación, identificando y relacionando los elementos propios del lenguaje de programación.
- RA 2: Escribir y probar clases aplicando los fundamentos de la programación orientada a objetos.
- RA3: Escribir y depurar código.
- RAs 4 y 7: Desarrollar programas organizados en clases.
- RA 5: Realizar operaciones de entrada y salida de información.

**Lenguaje de marcas** – Para el desarrollo del frontend web se emplea HTML5, CSS, JavaScript, utilizando React para la creación de interfaces dinámicas.

Resultados de aprendizaje que se cumplen:

- RA 1: Reconocer las características de lenguajes de marcas analizando e interpretando fragmentos de código.
- RA 2: Utilizar lenguajes de marcas para la presentación de información en la web.
- RA 3: Acceder y manipular documentos web mediante lenguajes de script de cliente.

**Entornos de desarrollo** – Durante la implementación del proyecto se utilizan distintos entornos integrados de desarrollo (IntelliJ IDEA y Visual Studio Code), así como herramientas de control de versiones.

Resultados de aprendizaje que se cumplen:

- RA 1: Reconocer los elementos y herramientas necesarias en el desarrollo de un programa informático.
- RA 3: Verificar el funcionamiento de programas diseñando y realizando pruebas.



Asignaturas del segundo curso:

**Acceso a datos** – El backend utiliza Hibernate como herramienta ORM para la gestión de la persistencia de datos en la base de datos.

Resultados de aprendizaje que se cumplen:

- RA 2: Desarrollar aplicaciones que gestionen información en bases de datos relacionales utilizando mecanismos de conexión.
- RA 3: Gestionar la persistencia de datos utilizando herramientas de mapeo objeto relacional.
- RA 6: Programar componentes de acceso a datos reutilizables y mantenibles.

**Desarrollo de interfaces** – El diseño de las interfaces web (React) y móviles (Flutter) aplica principios de usabilidad, accesibilidad y coherencia visual.

Resultados de aprendizaje que se cumplen:

- RA 1: Generar interfaces gráficas mediante editores visuales.
- RA 4: Diseñar interfaces aplicando criterios de usabilidad y accesibilidad.
- RA 8: Evaluar el funcionamiento de las interfaces realizando pruebas de usuario.

**Programación multimedia y dispositivos móviles** – El módulo se refleja en la creación de la aplicación móvil para clientes, desarrollada con Flutter y conectada al backend a través de servicios REST.

Resultados de aprendizaje que se cumplen:

- RA 1: Aplicar tecnologías de desarrollo para dispositivos móviles.
- RA 2: Desarrollar aplicaciones móviles empleando librerías y componentes específicos.
- RA 3: Integrar contenidos multimedia y elementos visuales interactivos en la interfaz móvil.

**Programación de servicios y procesos** – El backend ofrece servicios web REST que permiten la comunicación con la aplicación móvil y la web. Además, se implementan mecanismos de seguridad y gestión concurrente de peticiones.

Resultados de aprendizaje que se cumplen:

- RA 3: Programar mecanismos de comunicación en red mediante servicios web REST.
- RA 4: Desarrollar aplicaciones que ofrezcan servicios en red con criterios de eficiencia y disponibilidad.
- RA 5: Proteger los datos mediante autenticación y autorización con JWT.

**Proyecto de desarrollo de aplicaciones** – El proyecto integra las competencias adquiridas a lo largo del ciclo, abordando todas las fases del desarrollo de software: análisis, diseño, implementación, pruebas, despliegue y documentación.

Resultados de aprendizaje que se cumplen:

- RA 1: Identificar una necesidad del sector productivo y proponer una solución tecnológica adecuada.
- RA 2: Diseñar y planificar el proyecto, definiendo sus fases y documentación asociada.
- RA 3: Planificar la ejecución del proyecto estableciendo recursos, tiempos y responsabilidades.
- RA 4: Establecer mecanismos de seguimiento y control durante el desarrollo del proyecto.

**Sistemas de gestión empresarial** – El proyecto no implanta un ERP-CRM comercial, pero consiste en una plataforma de gestión empresarial propia que cubre funcionalidades como haría un CRM básico: gestión centralizada de clientes, servicios, empleados y citas, junto con un módulo de análisis de datos orientado a la toma de decisiones empresariales.

**Itinerario para la empleabilidad** – Durante el desarrollo del proyecto se ha realizado un estudio detallado de mercado y un análisis DAFO para identificar en qué estado se encuentra y qué posibles mejoras se pueden implementar.

## 1.2. Descripción del proyecto

### 1.2.1. Visión general

ReservApp es una plataforma diseñada para la gestión de reservas y servicios en pequeñas empresas. Su finalidad es ofrecer una solución que permita digitalizar la organización de citas, mejorar la comunicación entre empresas y clientes, y optimizar los procesos de atención mediante un entorno accesible y moderno.

El sistema se compone de dos aplicaciones principales:

- Aplicación web orientada a las empresas que ofrecen sus servicios.
- Aplicación móvil destinada a los clientes que desean reservar citas de forma rápida y sencilla.

Ambas aplicaciones se comunican con un backend común desarrollado en Java con Spring Boot, a través de una API RESTful.

### 1.2.2. Objetivos técnicos

- Implementar una arquitectura modular, escalable y mantenible.
- Garantizar la seguridad de los datos mediante autenticación, autorización y cifrado.
- Ofrecer interfaz de usuario intuitiva y adaptable a distintos dispositivos.
- Permitir la integración y despliegue ágil con Docker y Git.
- Asegurar la integridad y consistencia de los datos en la base de datos.

## 2. Estudio previo

En la actualidad, muchas empresas pequeñas siguen gestionando sus reservas manualmente, lo que puede generar ineficiencia y errores. La falta de digitalización limita su capacidad para ofrecer una atención moderna, rápida y organizada.

ReservApp se crea como una solución tecnológica que pretende cubrir esta necesidad, proporcionando una herramienta sencilla, centralizada y accesible para gestionar servicios y reservas.

### **Identificación del problema**

El problema principal es la ausencia de una plataforma digital integrada que permita a las pequeñas empresas administrar de forma eficiente sus horarios, servicios y citas, al mismo tiempo que ofrece a los usuarios una forma cómoda de reservar.

Principales inconvenientes del sistema tradicional:

- Pérdida de tiempo en la gestión manual de reservas.
- Difícil mantener actualizada la disponibilidad.
- Duplicidad o solapamiento de citas.
- Falta de visibilidad de los negocios locales.
- Ausencia de recordatorios o confirmaciones automáticas.

### **Objetivos del proyecto**

Objetivo principal: Desarrollar una plataforma digital que permita a pequeñas empresas y clientes gestionar reservas y servicios de forma eficiente, intuitiva y segura.

Objetivos específicos:

- Digitalizar el proceso de gestión de citas y servicios.
- Mejorar la comunicación entre empresa y cliente.
- Garantizar la integridad y coherencia de los datos mediante una base de datos relacional.
- Asegurar la escalabilidad y mantenibilidad del sistema mediante una arquitectura modular.
- Facilitar la implementación y despliegue mediante contenedores Docker.

### **Justificación del proyecto**

La transformación digital de los pequeños negocios es un proceso clave en la actualidad. Sin embargo, muchas pymes no cuentan con los recursos o conocimientos técnicos para adoptar soluciones.

ReservApp ofrece una alternativa accesible, de bajo coste y fácil implementación, que se adapta a distintos tipos de negocio.

## **2.1. Estudio de soluciones existentes**

Antes de iniciar el desarrollo del proyecto, se ha hecho un análisis de distintas plataformas y aplicaciones que ya están en el mercado y que ofrecen funciones similares.

El objetivo de este estudio es identificar sus principales características, ventajas, limitaciones y oportunidades de mejora que puedan aprovecharse en el desarrollo del proyecto.

### **2.1.1. Soluciones analizadas**

#### **2.1.1.1. Calendly**

Herramienta ampliamente utilizada para la programación de reuniones y citas. Permite a los usuarios compartir su disponibilidad y sincronizar calendarios con Google, Outlook, etc.

Ventajas:

- Interfaz profesional y sencilla.
- Integración con plataformas de videollamadas.
- Envío automático de recordatorios.

Desventajas:

- Orientada a profesionales individuales más que a negocios con múltiples servicios.
- Versión gratuita muy limitada.
- No dispone de app móvil para clientes.

#### **2.1.1.2. Booksy**

Plataforma enfocada a negocios del sector belleza, salud y bienestar. Ofrece gestión de reservas, pagos y reseñas de clientes.

Ventajas:

- Aplicación móvil muy completa y fácil de usar.
- Permite gestionar varios empleados y servicios.
- Incluye sistema de notificaciones y recordatorios.

Desventajas:

- Se necesita suscripción mensual.
- Exceso de funciones que pueden resultar complejas para pequeños negocios.

#### **2.1.1.3. Setmore**

Aplicación gratuita para la gestión de citas online. Ofrece integración con redes sociales y sitios web.

Ventajas:

- Dispone de versión web y aplicación móvil.
- Integración con calendarios externos.

Desventajas:

- Personalización limitada.
- Dependencia de la nube de Setmore.

#### **2.1.1.4. SimplyBook**

Sistema completo de reservas en línea para negocios con múltiples servicios. Incluye estadísticas, formularios personalizados y recordatorios automáticos.

Ventajas:

- Interfaz moderna y funcional.
- Soporte multilingüe y multimoneda.
- Amplia personalización.

Desventajas:

- Modelo de negocio basado en planes de pago.
- Complejidad de configuración.

### 2.1.2. Conclusiones

Del estudio realizado se concluye que, existen varias aplicaciones para la gestión de reservas, pero la mayoría:

- Son servicios cerrados en la nube, sin control sobre los datos ni posibilidad de personalización.
- Requieren pagos o suscripciones para acceder a funcionalidades avanzadas.
- Están enfocadas a sectores concretos o servicios online y no a una variedad de negocios en presencial.

## 3. Plan de trabajo

El plan de trabajo del proyecto, define de forma estructurada las fases necesarias para su desarrollo, estableciendo un orden lógico de ejecución, los objetivos de cada etapa y las tareas asociadas. Este enfoque permite organizar el tiempo y los recursos disponibles, facilitando el seguimiento del progreso y asegurando el cumplimiento de los objetivos planteados.

El proyecto se ha desarrollado siguiendo una metodología incremental, en la que cada fase se apoya en la anterior, permitiendo realizar pruebas y ajustes continuos.

Para la organización del proyecto se ha realizada una tabla Gantt con el trabajo a realizar por meses.

| Fase                                      | Oct | Nov | Dic | Ene | Feb | Mar | Abr | May |
|-------------------------------------------|-----|-----|-----|-----|-----|-----|-----|-----|
| Investigación y estudio previo            | X   |     |     |     |     |     |     |     |
| Diseño del sistema                        | X   | X   |     |     |     |     |     |     |
| Aprendizaje y desarrollo del backend      | X   | X   | X   |     |     |     |     |     |
| Diseño de mockups                         |     |     |     | X   |     |     |     |     |
| Aprendizaje y desarrollo del frontend web |     |     |     | X   | X   | X   |     |     |
| Aprendizaje y desarrollo de la app móvil  |     |     |     | X   | X   | X   | X   |     |
| Integración y Docker                      |     |     |     |     |     |     | X   | X   |
| Pruebas y validación                      |     |     |     |     |     |     | X   | X   |
| Documentación y memoria                   | X   | X   | X   | X   | X   | X   | X   | X   |

## **3.1. Fases del proyecto**

### **3.1.1. Análisis de requisitos**

En esta fase se identifican las necesidades del sistema y los requisitos funcionales y no funcionales del proyecto.

Tareas realizadas:

- Análisis del problema y del contexto de uso.
- Identificación de los perfiles de usuario.
- Definición de los requisitos funcionales.
- Definición de requisitos no funcionales.
- Estudio de soluciones existentes para detectar oportunidades de mejora.

Resultado: Documento base de requisitos y alcance del proyecto.

### **3.1.2. Diseño del sistema**

En esta fase se define la estructura técnica y lógica del sistema antes de comenzar la implementación.

Tareas realizadas:

- Diseño de la arquitectura multicapa del sistema.
- Definición de la comunicación entre frontend, backend y base de datos mediante API REST.
- Diseño del modelo de datos relacional y sus relaciones.
- Elaboración de diagramas entidad/relación.
- Diseño inicial de interfaces de usuario.

Resultado: Arquitectura definida y modelo de datos validado.



### 3.1.3. Implementación del backend

El backend constituye el núcleo funcional del sistema y se desarrolla utilizando Java con Spring Boot.

Tareas realizadas:

- Creación del proyecto backend.
- Implementación de entidades y repositorios con JPA/Hibernate.
- Desarrollo de la lógica de negocio en la capa de servicios.
- Creación de controladores REST para la comunicación con los clientes.
- Implementación de seguridad mediante Spring Security y autenticación con JWT.
- Pruebas unitarias básicas de los endpoints.

Resultado: API REST funcional y segura.

### 3.1.4. Desarrollo del frontend web

Esta fase se centra en la aplicación web destinada a las empresas.

Tareas realizadas:

- Creación de la estructura del proyecto frontend.
- Desarrollo de las vistas principales .
- Implementación de formularios para la gestión de empresas, servicios y horarios.
- Consumo de la API REST desde el frontend.
- Aplicación de diseño responsive mediante Bootstrap.
- Validación de datos en cliente.

Resultado: Aplicación web funcional e intuitiva para empresas.

### 3.1.5. Desarrollo de la aplicación móvil

En esta fase se desarrolla la aplicación móvil orientada a los clientes finales.

Tareas realizadas:

- Creación del proyecto en Flutter.
- Implementación de pantallas principales.
- Conexión con la API REST del backend.
- Gestión de sesiones de usuario.
- Pruebas de navegación y experiencia de usuario.

Resultado: Aplicación móvil operativa para la gestión de reservas.

### 3.1.6. Integración y despliegue

Una vez desarrollados todos los componentes, se procede a su integración y despliegue.

Tareas realizadas:

- Creación de contenedores para backend, frontend y base de datos mediante Docker.
- Configuración del archivo docker-compose.yml.
- Verificación de la comunicación entre servicios.
- Pruebas del sistema completo en entorno contenedorizado.

Resultado: Sistema desplegado de forma unificada y portable.

### 3.1.7. Pruebas y validación

Esta fase garantiza que el sistema cumple con los requisitos definidos.

Tareas realizadas:

- Pruebas funcionales de los principales casos de uso.
- Pruebas de integración entre frontend, backend y base de datos.
- Corrección de errores detectados.
- Validación de la experiencia de usuario.

Resultado: Sistema estable y validado.

### **3.1.8. Documentación y control de versiones**

La documentación y el control de versiones se realizan de forma continua durante todo el proyecto.

Tareas realizadas:

- Uso de GitHub para el control de versiones.
- Organización del repositorio por módulos.
- Elaboración de la memoria técnica del proyecto.
- Documentación del código y de la arquitectura.

Resultado: Proyecto documentado y mantenible.

## **4. Diseño**

La fase de diseño tiene como objetivo definir la estructura visual y funcional de la plataforma. Durante esta etapa se establecen la organización de las pantallas, la disposición de los elementos de la interfaz y la navegación entre las distintas secciones del sistema.

Para ello se elaboran mockups o prototipos visuales, que permiten representar de forma gráfica cómo será la aplicación tanto en su versión web como en la aplicación móvil. Estos prototipos sirven como guía durante el desarrollo y facilitan la detección de posibles mejoras en la experiencia de usuario antes de comenzar la programación.

### **4.1. Diseño de la interfaz de usuario**

El diseño de la interfaz se ha realizado teniendo en cuenta principios de usabilidad, simplicidad y accesibilidad, con el objetivo de ofrecer una experiencia intuitiva tanto para empresas como para clientes.

Entre los aspectos considerados en el diseño se encuentran:

- Organización clara de la información en cada pantalla.
- Navegación sencilla entre las distintas secciones de la aplicación.
- Uso de elementos visuales consistentes.

- Adaptación de la interfaz a diferentes dispositivos mediante diseño responsive.
- Reducción del número de pasos necesarios para realizar una reserva.

## 4.2. Creación de mockups

Vista principal para móvil

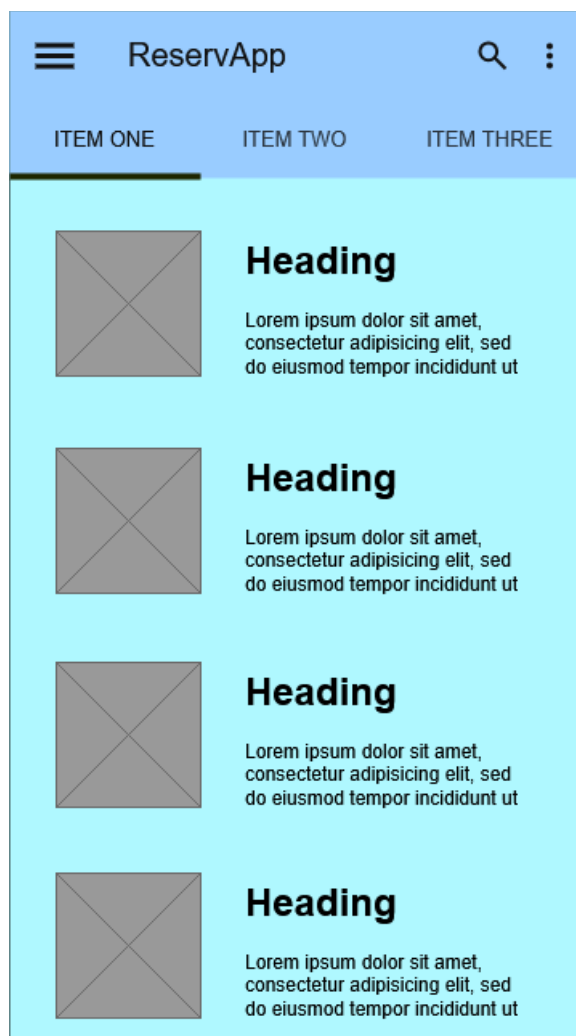


Fig 1. Mockup de la vista principal de la app

Vista detalle empresa para móvil

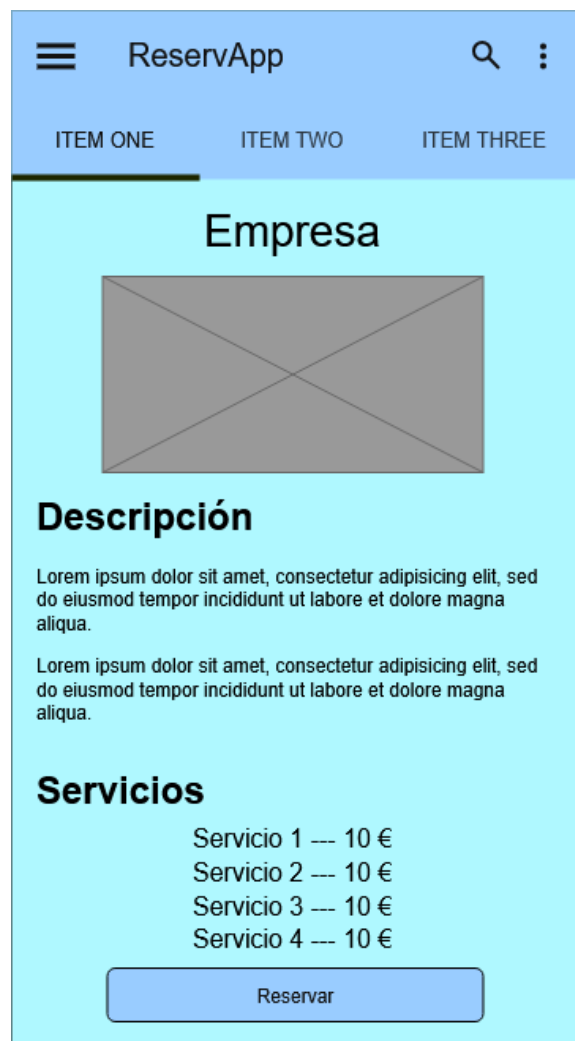


Fig 2. Mockup de la vista de detalle de la app

Vista web principal:

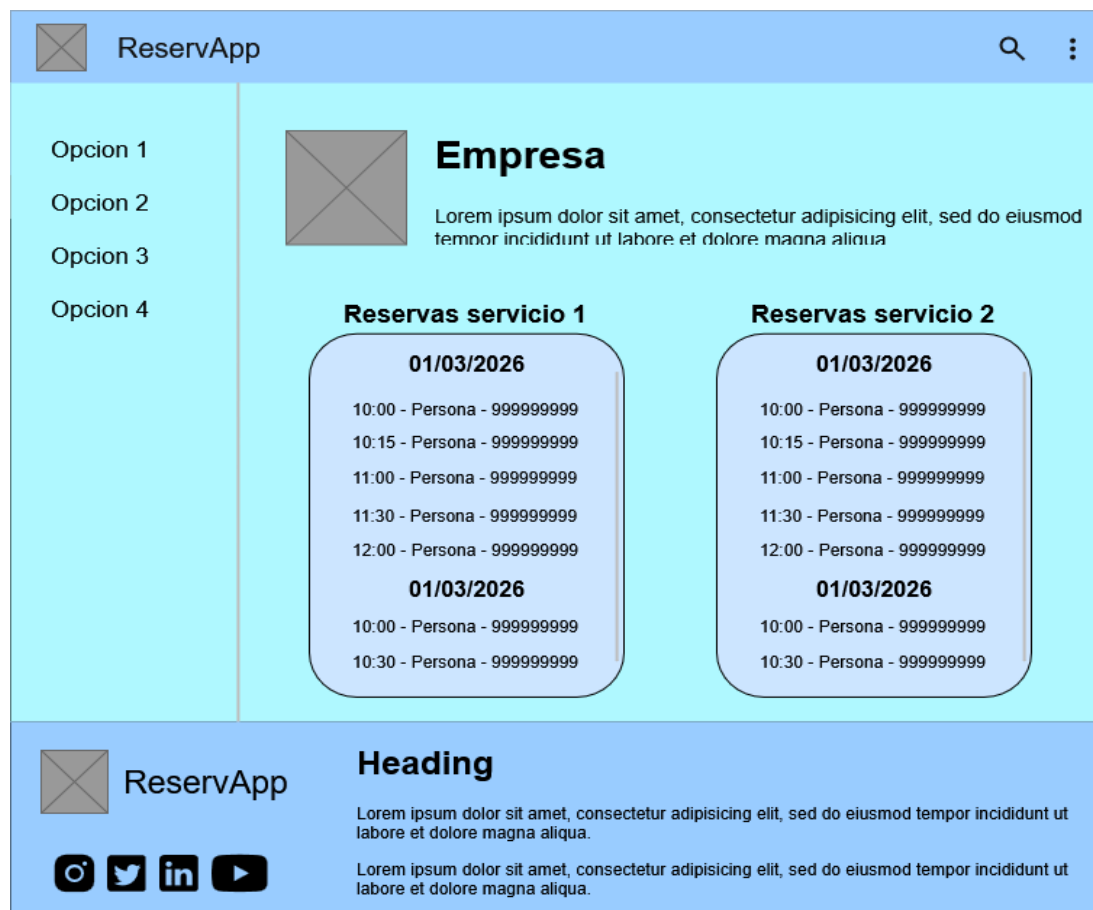


Fig 3. Mockup de la vista principal de la aplicación web

### 4.3. Herramientas de diseño

Para la creación de los prototipos se ha empleado la herramienta de diseño de interfaces Draw.io, que permiten diseñar interfaces de forma rápida.

El uso de esta herramienta facilita:

- La visualización del diseño final de la aplicación.
- La modificación rápida de elementos de la interfaz.
- La planificación del desarrollo del frontend.

## 4.4. Beneficios del prototipado

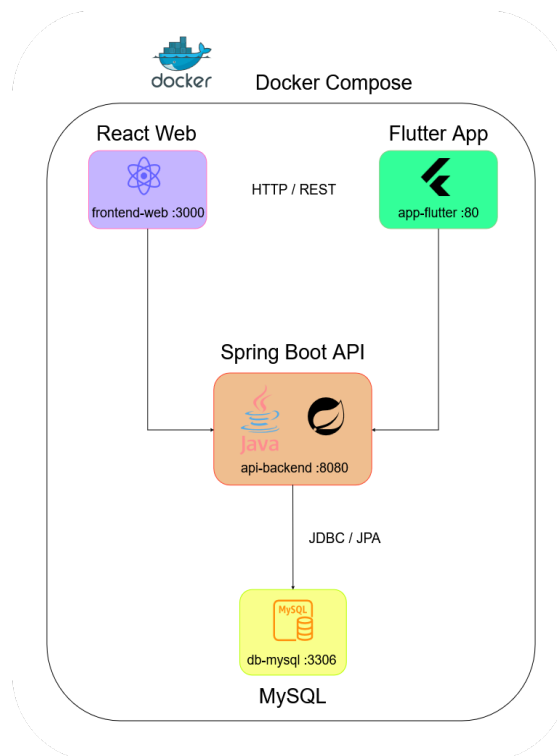
La creación de mockups antes de la fase de desarrollo ofrece varias ventajas:

- Permite detectar problemas de diseño de forma temprana.
- Facilita la planificación del desarrollo del frontend.
- Mejora la organización de las funcionalidades.
- Ayuda a mantener coherencia visual en toda la aplicación.

Gracias a esta fase de diseño, el desarrollo posterior de la plataforma se realiza de manera más estructurada y eficiente.

## 5. Implantación

La fase de implantación describe el proceso mediante el cual se ha desarrollado, configurado y desplegado el sistema ReservApp, tanto en el entorno del servidor como en los clientes.



**Fig 4.** Diagrama de flujo del proyecto

## 5.1. Implantación en el servidor

El servidor constituye el núcleo del sistema, ya que en él se ejecuta el backend, se gestiona la base de datos y se centraliza la lógica de negocio.

### 5.1.1. Desarrollo del backend

El backend ha sido desarrollado utilizando el framework Spring Boot, que permite crear aplicaciones web robustas y escalables basadas en Java.

Entorno de desarrollo utilizado:

- IDE: IntelliJ IDEA
- Lenguaje: Java
- Gestor de dependencias: Maven

Pasos seguidos:

- Creación del proyecto base con Spring Boot.
- Configuración de dependencias necesarias.

Definición de la estructura del proyecto en capas:

- Controladores
- Servicios
- Repositorios
- Implementación de entidades y relaciones.

Creación de endpoints REST para:

- Gestión de usuarios
- Empresas
- Servicios
- Reservas
- Empleados
- Implementación de seguridad mediante autenticación JWT.

Ejemplo de ejecución del backend:

- `mvn spring-boot:run`

El backend actúa como intermediario entre los clientes y la base de datos, procesando las peticiones y devolviendo respuestas en formato JSON.

#### **5.1.1.1. Estructura del backend**

El backend del sistema se ha organizado siguiendo una arquitectura multicapa, con el objetivo de separar responsabilidades, mejorar la mantenibilidad del código y facilitar su escalabilidad.

Esta estructura permite dividir la aplicación en distintos módulos, donde cada uno cumple una función específica dentro del sistema.

#### **Organización de paquetes**

El proyecto se estructura en los siguientes paquetes principales:

##### Controller

Contiene los controladores REST, encargados de gestionar las peticiones HTTP provenientes de los clientes.

Cada controlador está asociado a una entidad del sistema, por ejemplo:

- `AuthController` → autenticación de usuarios
- `EmpresaController` → gestión de empresas
- `ReservaController` → gestión de reservas

Responsabilidad:

- Recibir peticiones HTTP
- Validar datos de entrada
- Delegar la lógica en la capa de servicios
- Devolver respuestas en formato JSON

##### Service

Contiene la lógica de negocio de la aplicación.

Se divide en:

- Interfaces (`EmpresaService`, `ReservaService`, etc.)



- Implementaciones (EmpresaServiceImpl, ReservaServiceImpl, etc.)

Permite desacoplar la lógica de negocio de su implementación, facilitando cambios futuros y pruebas.

Responsabilidad:

- Aplicar reglas de negocio
- Validar operaciones
- Coordinar repositorios

### Repository

Gestiona el acceso a la base de datos mediante JPA/Hibernate.

Cada repositorio extiende de JpaRepository, lo que permite realizar operaciones CRUD sin necesidad de escribir SQL manual.

Ejemplo:

- UsuarioRepository
- ReservaRepository

Responsabilidad:

- Acceso directo a datos
- Consultas personalizadas

### Model

Contiene las entidades del sistema, que representan las tablas de la base de datos.

Ejemplos:

- Usuario
- Empresa
- Reserva
- Horario

Estas clases están anotadas con @Entity y definen las relaciones entre tablas.

### Mapper

Se encarga de transformar entidades en DTOs y viceversa.

Ejemplo:

- UsuarioMapper
- ReservaMapper

Evita exponer directamente las entidades de base de datos al cliente, mejorando la seguridad y la organización.

### Security

Gestiona la seguridad del sistema.

Componentes principales:

- JwtAuthenticationFilter → intercepta peticiones y valida tokens
- JwtService → genera y valida JWT
- SecurityConfig → configuración de seguridad

Funcionalidad:

- Autenticación de usuarios
- Protección de endpoints
- Control de acceso

### Config

Contiene configuraciones generales del sistema:

- PasswordEncoderConfig → cifrado de contraseñas
- WebConfig → configuración de CORS
- FileStorageService → gestión de archivos

### Exception

Define excepciones personalizadas:

- ResourceNotFoundException
- UnauthorizedException

Permite gestionar errores de forma controlada y devolver respuestas claras al cliente.

### **Flujo de una petición**

Cuando un cliente realiza una petición, el flujo es el siguiente:

1. El cliente envía una petición HTTP.
2. El controlador recibe la petición.
3. El controlador llama al servicio correspondiente.
4. El servicio aplica la lógica de negocio.
5. El servicio consulta o modifica datos a través del repositorio.
6. Se devuelve una respuesta al cliente en formato JSON.

### **5.1.2. Configuración de la base de datos**

Para la gestión de datos se ha utilizado MySQL.

Pasos realizados:

- Creación de la base de datos.
- Definición de tablas y relaciones.
- Configuración de conexión en el backend.

Ejemplo de configuración:

- `jdbc:mysql://localhost:3306/reservapp`
- `spring.datasource.username=usuario`
- `spring.datasource.password=contraseña`

La base de datos almacena toda la información del sistema, garantizando la integridad mediante claves primarias y foráneas.

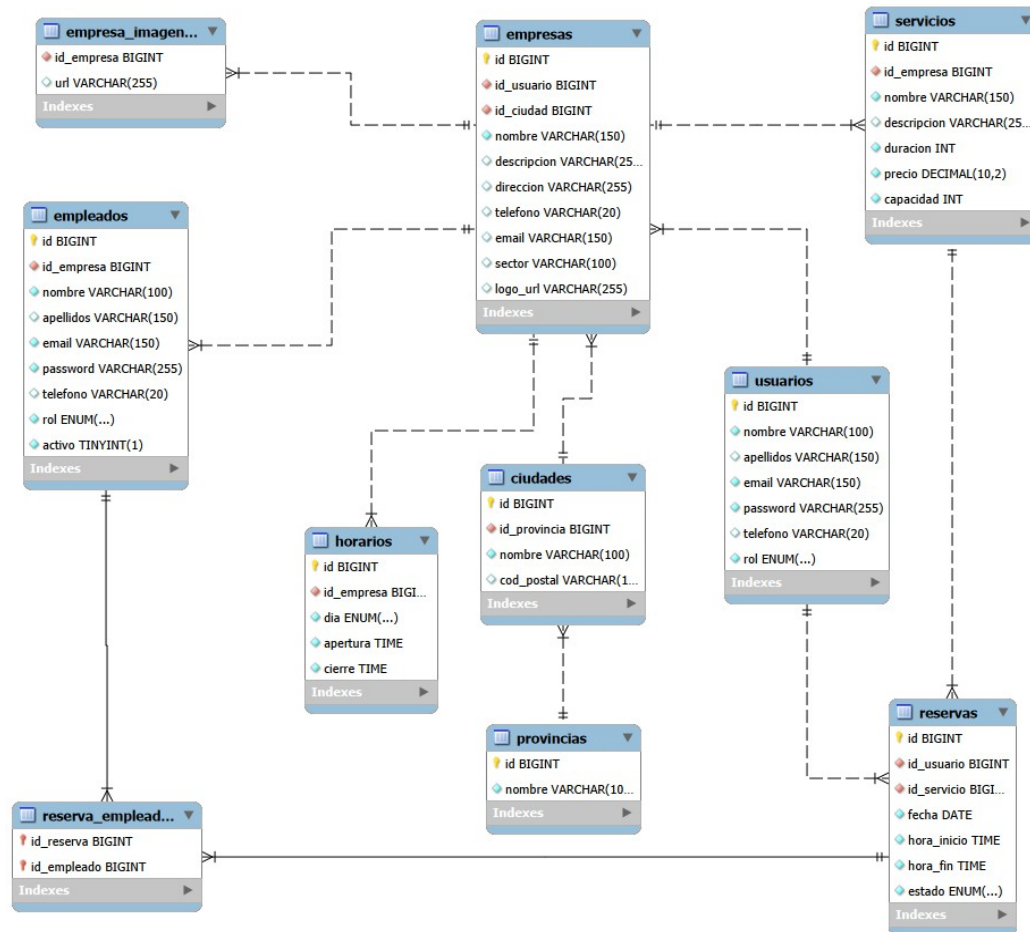


Fig 5. Diagrama entidad-relación de la base de datos.

### 5.1.3. Contenerización y despliegue

Para facilitar el despliegue se ha utilizado Docker, permitiendo ejecutar cada componente en un contenedor independiente.

Contenedores definidos:

- Backend
- Base de datos
- Frontend web
- Frontend móvil

Archivo principal:

- docker-compose.yml

Comando de ejecución:

- docker-compose up --build

Esto permite:

- Aislar servicios
- Facilitar la portabilidad
- Simplificar la instalación

## 5.2. Implantación del frontend web

La aplicación web de ReservApp está enfocada a las empresas que ofrecen sus servicios en la aplicación. A través de ella pueden gestionar su perfil, empleados, horarios, servicios y reservas desde un panel de administración accesible desde navegador.

Para su desarrollo se ha utilizado React como tecnología principal para la construcción de interfaces dinámicas, junto con Bootstrap para el diseño visual responsive y Recharts para la representación gráfica de estadísticas.

### 5.2.1. Entorno de desarrollo utilizado

Para la implementación del frontend se ha utilizado:

- IDE: Visual Studio Code
- Lenguaje: JavaScript
- Librería principal: React
- Bundler / servidor de desarrollo: Vite
- Navegación interna: React Router DOM
- Estilos: Bootstrap + CSS
- Gráficas: Recharts
- Control de calidad: ESLint
- Gestor de paquetes: npm

Comando de instalación inicial:

- npm install

Ejecución en entorno de desarrollo:

- npm run dev

### **5.2.2. Estructura del proyecto**

El proyecto frontend se ha organizado en carpetas independientes para facilitar el mantenimiento, reutilización de código y escalabilidad futura.

#### Assets

Contiene recursos estáticos utilizados en la aplicación, como pueden ser los logos.

Su función es centralizar todos los recursos gráficos del sistema.

#### Components

Incluye componentes reutilizables de React, permitiendo construir la interfaz mediante bloques modulares.

Se divide en subcarpetas:

#### **Common**

Contiene componentes genéricos reutilizables en distintas páginas, como:

- Ventana modal de confirmación
- Selector de empresas
- Animación de carga
- Mensajes de error
- EmptyState para cuando no hay información

#### **Layout**

Contiene componentes estructurales comunes a toda la web:

- Navbar
- Sidebar
- Layout

## **Contenedores principales**

Cada pestaña tiene sus componentes personalizados

### Context

Gestiona el estado global de la aplicación mediante React Context API.

Se utiliza para compartir información entre componentes sin necesidad de pasar propiedades manualmente.

Ejemplos:

- Usuario autenticado
- Token JWT
- Datos globales de sesión

### Hooks

Contiene hooks personalizados que encapsulan lógica reutilizable.

- Ejemplos:
- Peticiones HTTP
- Validaciones
- Control de autenticación

Esto mejora la limpieza y reutilización del código.

### Pages

Incluye las páginas principales de la aplicación, asociadas a rutas del sistema.

Ejemplos:

- Login
- Register
- Dashboard
- Reservas
- Servicios
- Horarios

Cada página combina distintos componentes reutilizables.

### Services

Contiene los módulos encargados de comunicarse con el backend mediante peticiones HTTP.

Ejemplos:

- authService.js
- empresaService.js
- reservaService.js

Utilizan fetch para consumir la API REST.

### Styles

Incluye hojas de estilo CSS personalizadas.

Se utilizan para complementar Bootstrap y adaptar la imagen visual del proyecto.

### Utils

Contiene funciones auxiliares reutilizables.

Ejemplos:

- Funciones para controlar los permisos de los usuarios

## **5.2.3. Funcionamiento general**

El frontend funciona como una single page application, donde la navegación entre secciones se realiza sin recargar la página completa.

El flujo es:

- El usuario accede al panel web.
- Introduce sus credenciales.
- Se envía la petición al backend.
- El backend devuelve un token JWT.
- El token se guarda en localStorage.
- El usuario accede a las funcionalidades.



#### **5.2.4. Ventajas de esta arquitectura frontend**

- Código modular y escalable.
- Reutilización de componentes.
- Mejor mantenimiento.
- Navegación rápida.
- Diseño adaptable a móviles y escritorio.
- Separación entre lógica, presentación y datos.

### **5.3. Implantación de la aplicación móvil**

Además de la aplicación web para las empresas, ReservApp tiene una aplicación móvil orientada a los clientes. Su objetivo es permitir la búsqueda de empresas, consulta de servicios disponibles y gestión de reservas desde dispositivos móviles de forma rápida, intuitiva y accesible.

Para su desarrollo se ha utilizado Flutter, framework multiplataforma que permite crear aplicaciones móviles nativas, utilizando Dart.

#### **5.3.1. Entorno de desarrollo utilizado**

Herramientas principales empleadas:

- Framework: Flutter
- Lenguaje: Dart
- IDE: Visual Studio Code
- Emulador Android
- Gestor de dependencias: pub

Instalación de dependencias:

- `flutter pub get`

Ejecución en entorno de desarrollo:

- `flutter run`

### 5.3.2. Estructura del proyecto móvil

La aplicación se ha organizado en carpetas independientes para mantener una arquitectura limpia y facilitar futuras ampliaciones.

#### Components

Contiene widgets reutilizables empleados en distintas pantallas.

- Header
- Cartas de las empresas

Esto permite mantener coherencia visual y reducir duplicidad de código.

#### Config

Incluye configuraciones globales de la aplicación.

Ejemplos:

- Colores
- Tamaños
- Elementos comunes
- URL base del backend

#### Models

Define las clases que representan las entidades de negocio recibidas desde la API.

Ejemplos:

- Usuario
- Empresa
- Servicio
- Reserva
- Horario

Estas clases facilitan convertir JSON en objetos Dart tipados.

### Screens

Contiene las pantallas principales de la aplicación.

Ejemplos:

- Auth
- Home
- Empresa

### Services

Gestiona la lógica de comunicación con el backend.

Ejemplos:

- AuthService
- ReservaService
- EmpresaService

Su función es centralizar peticiones HTTP y procesar respuestas.

### Utils

Donde se encuentra `api_client.dart`, encargado de centralizar las peticiones REST al servidor.

### **5.3.3. Cliente HTTP centralizado**

Para simplificar la conexión con el backend se ha desarrollado una clase común denominada `ApiClient`, encargada de gestionar todas las operaciones HTTP de la aplicación.

Esta clase implementa:

- Peticiones GET
- POST
- PUT
- PATCH
- DELETE
- Timeout de seguridad

- Inclusión automática del token JWT
- Funcionamiento

Antes de cada petición, el sistema obtiene el token almacenado localmente y lo añade a la cabecera Authorization.

'Authorization': 'Bearer \$token'

Esto permite acceder a endpoints protegidos sin necesidad de repetir código en cada pantalla.

### **Ventajas**

- Reutilización de código
- Mantenimiento centralizado
- Mayor seguridad
- Código más limpio
- Escalabilidad futura

### **5.3.4. Funcionalidades principales implementadas**

La aplicación móvil permite al cliente:

- Registro e inicio de sesión
- Búsqueda de empresas
- Consulta de servicios disponibles
- Selección de fecha y hora
- Realización de reservas
- Consulta del historial de citas
- Cancelación de reservas

### 5.3.5. Ventajas técnicas de Flutter

La utilización de Flutter aporta los siguientes beneficios:

- Alto rendimiento nativo
- Desarrollo multiplataforma
- Interfaz moderna y fluida
- Gran productividad de desarrollo
- Escalabilidad futura a iOS
- Amplia comunidad y soporte

## 6. Análisis DAFO

Con el objetivo de evaluar la viabilidad y situación del proyecto ReservApp desde una perspectiva empresarial y estratégica, se ha realizado un análisis DAFO.

Este análisis permite identificar los factores internos y externos que pueden influir en el desarrollo y crecimiento de la plataforma.

| Fortalezas                                               | Debilidades                                                      |
|----------------------------------------------------------|------------------------------------------------------------------|
| Plataforma multiplataforma con aplicación web y móvil.   | Proyecto desarrollado individualmente.                           |
| Arquitectura moderna basada en API REST y Docker.        | Recursos económicos y de infraestructura limitados.              |
| Interfaz intuitiva y adaptable a distintos dispositivos. | Dependencia de servicios externos para futuro escalado.          |
| Código modular y escalable.                              | Falta de experiencia comercial en lanzamiento de productos SaaS. |
| Sistema centralizado de gestión de reservas.             | Necesidad de mantenimiento continuo y soporte técnico.           |
| Uso de tecnologías ampliamente demandadas en el mercado. | Competencia elevada en el sector de reservas online.             |

| Oportunidades                                                        | Amenazas                                                       |
|----------------------------------------------------------------------|----------------------------------------------------------------|
| Crecimiento de la digitalización en pequeñas empresas.               | Existencia de plataformas consolidadas como Calendly o Booksy. |
| Aumento del uso de aplicaciones móviles para reservas.               | Cambios rápidos en tecnologías y tendencias digitales.         |
| Posibilidad de implementar modelo SaaS por suscripción.              | Riesgos relacionados con ciberseguridad y protección de datos. |
| Integración futura con pagos online y estadísticas avanzadas.        | Dependencia de la conexión a internet y servidores.            |
| Expansión a distintos sectores empresariales.                        | Costes futuros de alojamiento y escalabilidad.                 |
| Crecimiento de negocios locales que buscan digitalización accesible. | Aparición de nuevas aplicaciones competidoras.                 |

## 6.1. Conclusiones del análisis

El análisis DAFO muestra que ReservApp presenta una base tecnológica sólida y adaptable, especialmente orientada a pequeñas empresas que desean digitalizar la gestión de sus servicios y reservas.

Las principales fortalezas del proyecto se encuentran en su arquitectura moderna, su accesibilidad multiplataforma y la posibilidad de crecimiento futuro mediante nuevas funcionalidades.

Por otro lado, las principales amenazas provienen de la fuerte competencia existente en el sector y de la necesidad de mantener el sistema actualizado y seguro.

Aun así, el crecimiento de la transformación digital en pequeños negocios representa una oportunidad favorable para el desarrollo y evolución de la plataforma.

## 7. Recursos

Para el desarrollo e implantación del proyecto ReservApp se han utilizado distintos recursos hardware y software, así como diferentes herramientas de desarrollo y entornos de ejecución necesarios para implementar tanto el backend como las aplicaciones cliente.

### 7.1. Herramientas hardware

El desarrollo del proyecto se ha realizado utilizando un equipo de sobremesa con capacidad suficiente para ejecutar simultáneamente entornos de desarrollo, emuladores móviles, contenedores Docker y servicios de base de datos.

Recursos hardware utilizados:

- Ordenador sobremesa
  - Procesador AMD Ryzen 7 3700X 8-Core, 3.59GHz
  - Memoria RAM de 32GB

El equipo utilizado ha permitido:

- Ejecutar el backend desarrollado en Java con Spring Boot.
- Levantar contenedores mediante Docker.
- Compilar y ejecutar aplicaciones React y Flutter.
- Ejecutar servidores de base de datos.
- Realizar pruebas simultáneas entre cliente y servidor.

## 7.2. Herramientas software

Durante el desarrollo del proyecto se han empleado distintas herramientas especializadas según la función requerida.

| Herramienta        | Función                                        |
|--------------------|------------------------------------------------|
| IntelliJ IDEA      | Desarrollo del backend con Spring Boot         |
| Visual Studio Code | Desarrollo frontend React y aplicación Flutter |
| Docker             | Contenerización y despliegue                   |
| MySQL Workbench    | Administración y modelado de base de datos     |
| GitHub             | Control de versiones                           |
| Postman            | Pruebas de endpoints REST                      |
| Flutter            | Desarrollo de aplicación móvil                 |
| React              | Desarrollo del frontend web                    |
| Vite               | Compilación y servidor de desarrollo React     |

### **7.3. Sistemas operativos empleados**

El proyecto se ha desarrollado y probado sobre sistemas operativos compatibles con las herramientas utilizadas.

Sistemas utilizados:

- Windows 10
- Android
- Entornos Linux contenidos mediante Docker

### **7.4. Paquetes y dependencias necesarias**

Para el correcto funcionamiento del sistema ha sido necesario instalar diferentes paquetes y dependencias.

#### **Backend**

Tecnologías y dependencias principales:

- Java JDK
- Maven
- Spring Boot
- Spring Security
- JPA / Hibernate
- JWT
- Driver MySQL
- Lombok
- MapStruct
- SpringDoc OpenAPI



### **Frontend web**

Dependencias instaladas mediante npm:

- React
- React Router DOM
- Bootstrap
- Recharts
- Vite

### **Aplicación móvil**

Dependencias Flutter utilizadas:

- http
- shared\_preferences

### **Contenedorización**

Para el despliegue del sistema se instalaron:

- Docker Engine
- Docker Compose

## **8. Conclusiones**

El desarrollo de ReservApp ha permitido aplicar de forma práctica los conocimientos adquiridos durante el curso, integrando tecnologías de backend, frontend web, aplicaciones móviles, bases de datos y despliegue mediante contenedores.

El proyecto ha supuesto la creación de una plataforma funcional orientada a la gestión de reservas y servicios para pequeñas empresas, ofreciendo una solución moderna, accesible y escalable.

Además del aprendizaje técnico, el desarrollo del proyecto ha permitido trabajar aspectos relacionados con la planificación, organización del trabajo, resolución de problemas y diseño de arquitecturas software reales.

## 8.1. Grado de consecución de objetivos

Se han alcanzado los siguientes objetivos:

### **Desarrollo de una plataforma funcional**

Se ha implementado un sistema completo con:

- Backend desarrollado con Java y Spring Boot.
- Aplicación web desarrollada con React.
- Aplicación móvil desarrollada con Flutter.
- Base de datos relacional con MySQL.

### **Digitalización del proceso de reservas**

El sistema permite a las empresas:

- Gestionar servicios.
- Organizar horarios.
- Administrar reservas.
- Gestionar empleados.

Y a los clientes:

- Buscar empresas.
- Reservar citas.
- Consultar reservas realizadas.

### **Implementación de una arquitectura moderna**

Se ha desarrollado una arquitectura desacoplada basada en API REST.

Además, se ha implementado:

- Seguridad mediante JWT.
- Persistencia de datos mediante JPA/Hibernate.
- Despliegue mediante Docker.

### **Aplicación de buenas prácticas**

Durante el desarrollo se han aplicado principios de:

- Modularidad.
- Separación de responsabilidades.
- Reutilización de componentes.
- Organización por capas.
- Control de versiones con Git.

## **8.2. Problemas encontrados**

Durante el desarrollo del proyecto se han encontrado diversas dificultades que han requerido investigación y adaptación.

Principales problemas encontrados:

- Realizar un sistema de autenticación único tanto para usuarios como para empleados, que utiliza el mismo endpoint `/auth/login`, que devuelve información distinta según el tipo.
- La lógica de disponibilidad de horas, que tuvo que implementarse en ambos frontends de forma independiente.
- La configuración de CORS para que el frontend pudiera comunicarse con el backend.
- La subida y gestión de imágenes con almacenamiento local.

## **8.3. Mejoras**

Aunque el sistema desarrollado es funcional, existen muchas posibilidades de ampliación y mejora.

- Sistema de pagos online

- Integración con plataformas de pago para permitir reservas pagadas directamente desde la aplicación.

- Sistema de valoraciones y reseñas

- Permitir que los clientes valoren servicios y empresas tras cada reserva.

- Notificaciones automáticas
  - Correo electrónico
  - Notificaciones
- Panel avanzado de estadísticas
  - Ampliación del dashboard empresarial.
- Geolocalización y búsqueda avanzada
  - Implementar filtros por proximidad y mapas interactivos para localizar empresas cercanas.

## 9. Bibliografía

### Spring Boot y Java:

<https://docs.oracle.com/en/java/javase/17/>  
<https://spring.io/projects/spring-boot>  
<https://spring.io/projects/spring-security>  
<https://jakarta.ee/learn/>  
<https://hibernate.org/orm/documentation/7.3/>  
<https://projectlombok.org/features/>  
<https://mapstruct.org/documentation/installation/>  
<https://maven.apache.org/guides/>

### Frontend web:

<https://react.dev/learn>  
<https://scrimba.com/learn-react-c0e>  
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>  
<https://developer.mozilla.org/es/docs/Web/CSS>  
<https://recharts.github.io/en-US/guide/>  
<https://getbootstrap.com/docs/5.3/getting-started/introduction/>  
<https://vite.dev/guide/>  
<https://reactrouter.com/home>  
<https://es.react.dev/reference/react>

### Flutter y Dart:

<https://docs.flutter.dev/>  
<https://dart.dev/guides>  
<https://pub.dev/>

**Otros:**

<https://stackoverflow.com/questions>

<https://github.com/>

<https://git-scm.com/docs>

<https://docs.docker.com/>

<https://hub.docker.com/>

**Repositorios:**

Proyecto entero: [https://github.com/JordiCosM/proyecto\\_2DAM\\_Jordi](https://github.com/JordiCosM/proyecto_2DAM_Jordi)

Backend: <https://github.com/JordiCosM/reservapp-backend>

Frontend web: [https://github.com/JordiCosM/reservapp\\_web](https://github.com/JordiCosM/reservapp_web)

Frontend móvil: [https://github.com/JordiCosM/reservapp\\_mobile](https://github.com/JordiCosM/reservapp_mobile)